



浙江大學
ZHEJIANG UNIVERSITY

智能物联网系统设计实验报告

作业一硬件基础实验

姓名 _____

学号 _____

院系 信息与电子工程学院

2026 年 5 月 6 日

摘要

本实验报告基于 ESP32 开发板与 Arduino IDE，完成了三项基础的传感器与执行器实验。主要包括：1. 利用 ESP32 的 ADC 功能读取光敏电阻的模拟量与电压值；2. 通过 I2C 通信协议读取 BMP280 传感器的环境温度与气压信息；3. 利用 PWM（脉冲宽度调制）技术控制 RGB 三色 LED 灯，实现不同颜色的平滑混合与渐变。报告详细记录了各实验的实现原理、代码分析以及实验结果。

目录

1	实验一：读取光敏电阻的读数	3
1.1	实验目的与过程	3
1.2	代码实现与原理解析	3
1.3	实验效果图片	4
2	实验二：读取 BMP280 传感器的温度信息	5
2.1	实验目的与过程	5
2.2	代码实现与原理解析	5
2.3	实验效果图片	6
3	实验三：将三色小灯混合出不同的颜色	8
3.1	实验目的与过程	8
3.2	代码实现与原理解析	8
3.3	实验效果图片	9
4	实验总结	11

1 实验一：读取光敏电阻的读数

1.1 实验目的与过程

本实验旨在了解并掌握 ESP32 的模数转换（ADC）功能。光敏电阻的阻值会随着光照强度的变化而变化，通过搭建电路将光照的变化转化为电压的变化。实验过程如下：

1. 将光敏电阻电路接入 ESP32 的 Pin 32 引脚。
2. 在 Arduino IDE 中编写程序，初始化串口。
3. 配置 ADC 的分辨率，并在循环中持续读取模拟值与等效毫伏电压值，通过串口打印输出。
4. 打开串口监视器，遮挡或使用强光照射光敏电阻，观察读数变化。

1.2 代码实现与原理解析

在示例代码基础上，由于 ESP32 的默认 ADC 分辨率为 12 位（0-4095），将 `analogReadResolution()` 的参数修正为 12。

```
1 void setup() {  
2     // 初始化串口通信，波特率 115200  
3     Serial.begin(115200);  
4  
5     // 设置 ADC 分辨率为 12 位（范围 0-4095）  
6     analogReadResolution(12);  
7 }  
8  
9 void loop() {  
10    // 读取引脚 32 的模拟值  
11    int analogValue = analogRead(32);  
12    // 读取引脚 32 的等效毫伏值  
13    int analogVolts = analogReadMilliVolts(32);  
14  
15    // 打印读取到的数值  
16    Serial.printf("ADC analog value = %d\n", analogValue);  
17    Serial.printf("ADC millivolts value = %d\n", analogVolts);  
18  
19    // 延时 1 秒，避免刷新过快影响阅读  
20    delay(1000);  
21 }
```

Code Listing 1: 光敏电阻读取代码 (AnalogRead.ino)

代码原理解析：程序通过 `analogRead(32)` 指令调用 ESP32 内部的 ADC 模块，将引脚上的模拟电压转换为离散的数字量。分辨率设为 12 位意味着 3.3V 的电压被分成了 4096 份。同时，`analogReadMilliVolts()` 是 ESP32 特有的 API，它内部包含了 ADC 的校准曲线，能更准确地将 ADC 读数还原为实际的毫伏 (mV) 电压值。

1.3 实验效果图片



```
输出  串口监视器 ×
消息 (按回车将消息发送到“COM5”上的“ESP32 Dev Module”)
ADC analog value = 3378
ADC millivolts value = 2787
ADC analog value = 3428
ADC millivolts value = 2815
ADC analog value = 3358
ADC millivolts value = 2774
ADC analog value = 3335
ADC millivolts value = 2763
ADC analog value = 3348
ADC millivolts value = 2771
ADC analog value = 3376
ADC millivolts value = 2786
ADC analog value = 3377
ADC millivolts value = 2790
```

图 1: 遮挡光敏电阻后串口读数与电压值变化



```
输出  串口监视器 ×
消息 (按回车将消息发送到“COM5”上的“ESP32 Dev Module”)
ADC analog value = 27
ADC millivolts value = 160
ADC analog value = 29
ADC millivolts value = 165
ADC analog value = 27
ADC millivolts value = 162
ADC analog value = 16
ADC millivolts value = 160
ADC analog value = 29
ADC millivolts value = 168
ADC analog value = 27
ADC millivolts value = 165
ADC analog value = 30
ADC millivolts value = 165
```

图 2: 用手电筒照射光敏电阻后串口读数与电压值变化

2 实验二：读取 BMP280 传感器的温度信息

2.1 实验目的与过程

本实验使用 BMP280 环境传感器，通过 I2C 总线读取当前的温度、气压及计算出的海拔高度。实验过程：

1. 确认 BMP280 模块的 VCC, GND 连接正确，SDA 接 ESP32 的 Pin 21，SCL 接 Pin 22。
2. 在 Arduino IDE 中安装 Adafruit_BMP280 库。
3. 编译并烧录程序，打开串口监视器，观察温度及气压信息输出。

2.2 代码实现与原理解析

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>
4
5 #define BMP_SDA 21 // I2C SDA 引脚
6 #define BMP_SCL 22 // I2C SCL 引脚
7 Adafruit_BMP280 bmp280;
8
9 void setup() {
10     Serial.begin(115200);
11     Serial.println("Initializing BMP280...");
12
13     // 初始化 I2C 通信, BMP280 常见地址为 0x76 或 0x77
14     boolean status = bmp280.begin(0x76);
15     if (!status) {
16         Serial.println("Could not find a valid BMP280 sensor, check
17             wiring!");
18         while (1);
19     }
20 }
21
22 void loop() {
23     float temp = bmp280.readTemperature(); // 读取温度值, 单位摄氏度
24     float press = bmp280.readPressure() / 100.0F; // 读取大气压并转换为
25     hPa
```

```
24 float altitude = bmp280.readAltitude(1013.25); // 读取估计海拔(以标
    准大气压为基准)
25
26 Serial.print("Temperature: ");
27 Serial.print(temp);
28 Serial.println(" *C");
29
30 Serial.print("Pressure: ");
31 Serial.print(press);
32 Serial.println(" hPa");
33
34 Serial.print("Altitude: ");
35 Serial.print(altitude);
36 Serial.println(" m");
37
38 Serial.println("-----");
39 delay(2000); // 每两秒刷新一次
40 }
```

Code Listing 2: BMP280 传感器读取代码 (ESP32_BMP280.ino)

代码原理解析：BMP280 内部集成了 ADC 与温度补偿算法。代码中 `bmp280.begin(0x76)` 初始化了 I2C 总线，并配置了传感器的寄存器。`readTemperature()` 函数通过向特定的 I2C 地址发送指令，读取传感器的原始温度数据，随后调用库内部的校准矩阵进行多项式计算，最终输出摄氏度数据。

2.3 实验效果图片

```

输出  串口监视器  ×
消息 (按回车将消息发送到“COM5”上的“ESP32 Dev Module”)

-----
Temperature: 19.03 *C
Pressure: 1024.27 hPa
Altitude: -91.35 m
-----
Temperature: 19.03 *C
Pressure: 1024.28 hPa
Altitude: -91.42 m
-----
Temperature: 19.01 *C
Pressure: 1024.27 hPa
Altitude: -91.33 m
-----

```

图 3: BMP280 传感器环境数据读取结果

```

输出  串口监视器  ×
消息 (按回车将消息发送到“COM5”上的“ESP32 Dev Module”)

-----
Temperature: 31.13 *C
Pressure: 1032.62 hPa
Altitude: -160.04 m
-----
Temperature: 31.32 *C
Pressure: 1033.67 hPa
Altitude: -168.60 m
-----
Temperature: 31.45 *C
Pressure: 1034.56 hPa
Altitude: -175.94 m
-----

```

图 4: 改变 BMP280 传感器环境温度后数据读取结果

3 实验三：将三色小灯混合出不同的颜色

3.1 实验目的与过程

传统的 LED 只能发出单色光，而 RGB LED 内部封装了红、绿、蓝三个发光二极管。本实验旨在利用 ESP32 的 PWM 功能，通过改变三原色各自的亮度比例，实现丰富的颜色混合。我稍改编了示例代码，增加了颜色交叉渐变逻辑。

3.2 代码实现与原理解析

```
1 #define RED_PIN 18
2 #define GREEN_PIN 17
3 #define BLUE_PIN 16
4 // 经测试，示例程序中颜色引脚定义有误，颠倒了红、蓝两引脚数字
5
6 void setup() {
7     Serial.begin(115200);
8     pinMode(RED_PIN, OUTPUT);
9     pinMode(GREEN_PIN, OUTPUT);
10    pinMode(BLUE_PIN, OUTPUT);
11 }
12
13 // 封装颜色设置函数
14 void setColor(int redValue, int greenValue, int blueValue) {
15     analogWrite(RED_PIN, redValue);
16     analogWrite(GREEN_PIN, greenValue);
17     analogWrite(BLUE_PIN, blueValue);
18 }
19
20 void loop() {
21     // 1. 从红色渐变到黄色 (红+绿)
22     for (int i = 0; i <= 255; i++) {
23         setColor(255, i, 0);
24         delay(10);
25     }
26     // 2. 从黄色渐变到绿色 (减去红色)
27     for (int i = 255; i >= 0; i--) {
28         setColor(i, 255, 0);
29         delay(10);
30     }
```

```
31 // 3. 从绿色渐变到青色 (绿+蓝)
32 for (int i = 0; i <= 255; i++) {
33     setColor(0, 255, i);
34     delay(10);
35 }
36 // 4. 从青色渐变到蓝色 (减去绿色)
37 for (int i = 255; i >= 0; i--) {
38     setColor(0, i, 255);
39     delay(10);
40 }
41 // 5. 从蓝色渐变到紫色 (蓝+红)
42 for (int i = 0; i <= 255; i++) {
43     setColor(i, 0, 255);
44     delay(10);
45 }
46 // 6. 从紫色渐变回红色 (减去蓝色)
47 for (int i = 255; i >= 0; i--) {
48     setColor(255, 0, i);
49     delay(10);
50 }
51 }
```

Code Listing 3: RGB 颜色混合与渐变代码 (RGB_Mixing.ino)

代码原理解析：该程序运用了加色法原理。在 `setColor` 函数中，`analogWrite()` 输出了占空比从 0 到 255 的 PWM 方波。PWM 波的占空比决定了 LED 的平均驱动电流，进而控制单色光的亮度。例如，当红色通道保持 255（最亮），绿色通道从 0 递增至 255 时，人眼观察到的颜色就会从纯红平滑地过渡到橙色，最终混合成黄色。通过嵌套的 `for` 循环，代码基本实现了覆盖全色的呼吸灯效果。

3.3 实验效果图片

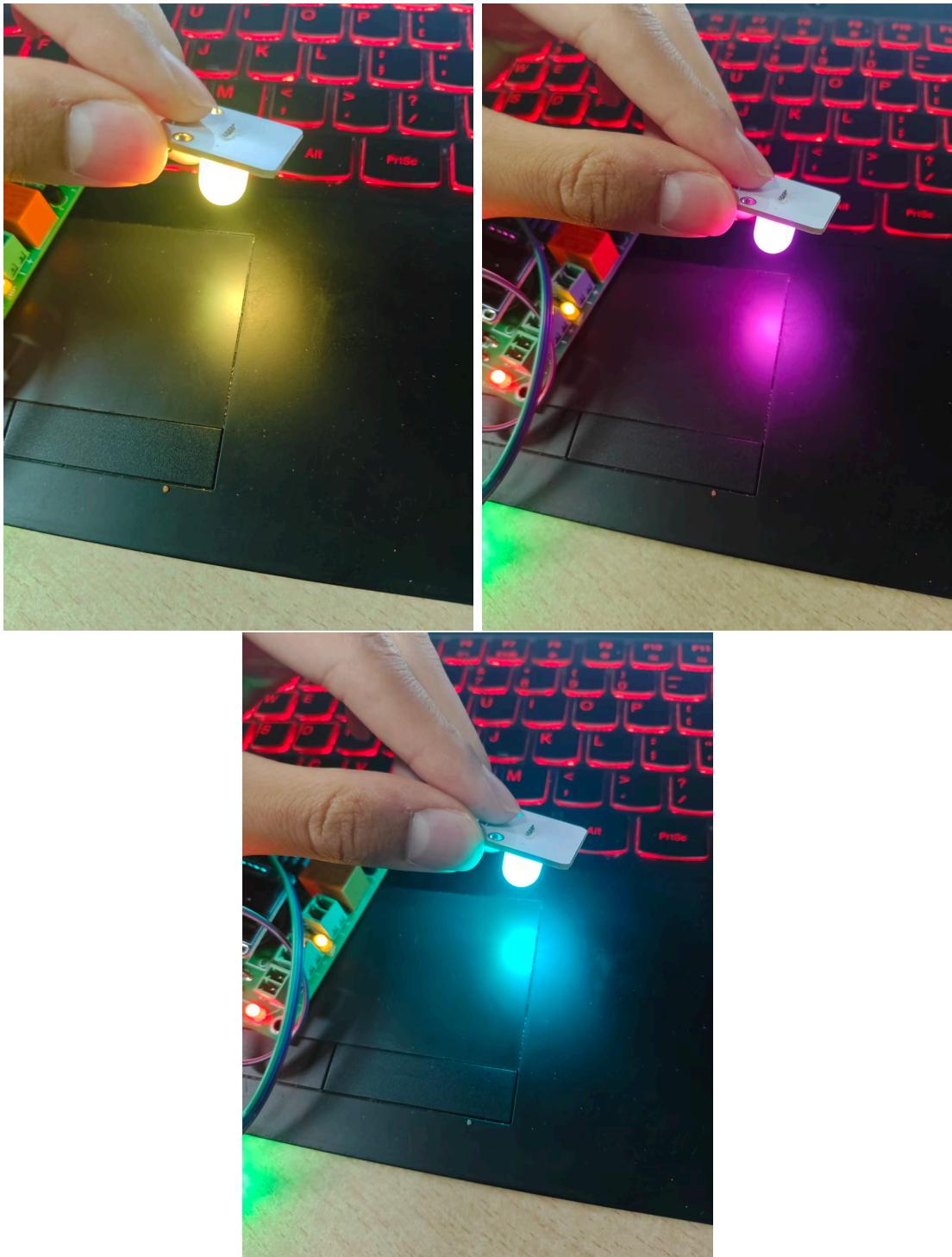


图 5: RGB 三色小灯混合出不同颜色的实际效果

4 实验总结

通过本次实验，我不仅熟悉了 Arduino IDE 的基础操作与 ESP32 开发板的引脚特性，还深入理解了底层的数据采集与硬件控制逻辑。光敏实验让我直观地理解了模数转换的过程与分压电路的运用；BMP280 实验让我掌握了 I2C 总线的初始化与复杂传感器库的调用方法；而在 RGB 调色实验中，我将简单的单色点亮优化为了基于 PWM 占空比的动态色彩混合，加深了对代码控制物理现象的认识。